University Enrollment System

A Deep-Dive Technical Explainer

Prepared for Salman Adnan's portfolio

This document exists to make every part of this project understandable to someone with no prior exposure to it, technical term by technical term. It is a working reference for the owner's own understanding and interview/defense preparation — it is not part of the published portfolio website. Everything stated here comes directly from the project's source code, its `README.md`*, and its* `Proposal.pdf`*. Anything not directly confirmed by those sources is explicitly marked as* ***[inferred]*** *or* ***[flagged]****.*

Contents

1 What this project is

The **University Enrollment System** is a desktop application that lets a student log in, verify their identity with an emailed one-time code, and then enroll in, add, drop, or swap university course sections, view their weekly class schedule, mark attendance, and review a log of everything they have done. It was built as the semester project for the Databases course (course codes CS/CE 355/373) at Habib University, in the students' 4th semester. It is a two-person team project: **Salman Adnan** and **Shayan Wasif**. Per the project's own README, Salman's specific areas of the build were the email verification (MFA) flow, the attendance module and its email confirmations, the Docker packaging, and integration/bug-fixing work that tied the separately-built screens together.

1.1 Why it exists — the motivating proposal

The repository includes a short project proposal (`Proposal.pdf`) that frames the motivation: Habib University's real student-records system is ***PeopleSoft Campus Solutions (PSCS)***, a commercial enterprise platform used for enrollment, attendance, and academic records. The proposal identifies three concrete gaps in that real system and pitches this project as a lightweight prototype addressing them:

- **No multi-factor authentication.** PSCS logins were password-only; the proposal calls for OTP-based (one-time password) verification as a second login step.
- **No attendance notifications.** Instructors mark absences in PSCS, but students only find out by manually checking; the proposal calls for automated email/SMS alerts.
- **No activity history.** Students swap/add/drop courses under enrollment deadlines with no record of what they changed; the proposal calls for a detailed activity log.

This project implements working versions of all three ideas: a six-digit email code after password login (Section 6), an attendance screen that emails a confirmation on every change (Section 12), and an activity log table that records every add, drop, swap, and attendance change (Section 13). The proposal only mentions SMS as a possibility alongside email; [**flagged**] the shipped code implements email notifications only — no SMS integration exists in the source.

1.2 Beginner primer: framing concepts

Before diving into files, four concepts recur throughout this document and are defined once here so they are not re-explained at every mention:

GUI framework — a code library that gives a program windows, buttons, text boxes and other visual controls, and a way to react to clicks and typing. This project uses ***PyQt6***, the Python bindings for the Qt6 GUI toolkit (a widely used C++ framework for building desktop application interfaces).

ORM (Object-Relational Mapper) — a library that lets code manipulate database rows as if they were ordinary programming-language objects, instead of writing raw SQL by hand. This project uses ***SQLAlchemy*** as its ORM, mapping Python classes such as `Student` and `Course` onto tables in a database file.

SQLite — a lightweight, file-based relational database engine. Unlike database servers such as PostgreSQL or MySQL, an SQLite database is a single file on disk (`student.db` in this project) with no separate server process to run.

SMTP (Simple Mail Transfer Protocol) — the standard protocol used to send email. Python's built-in `smtplib` module implements an SMTP client; this project uses it to send both the login

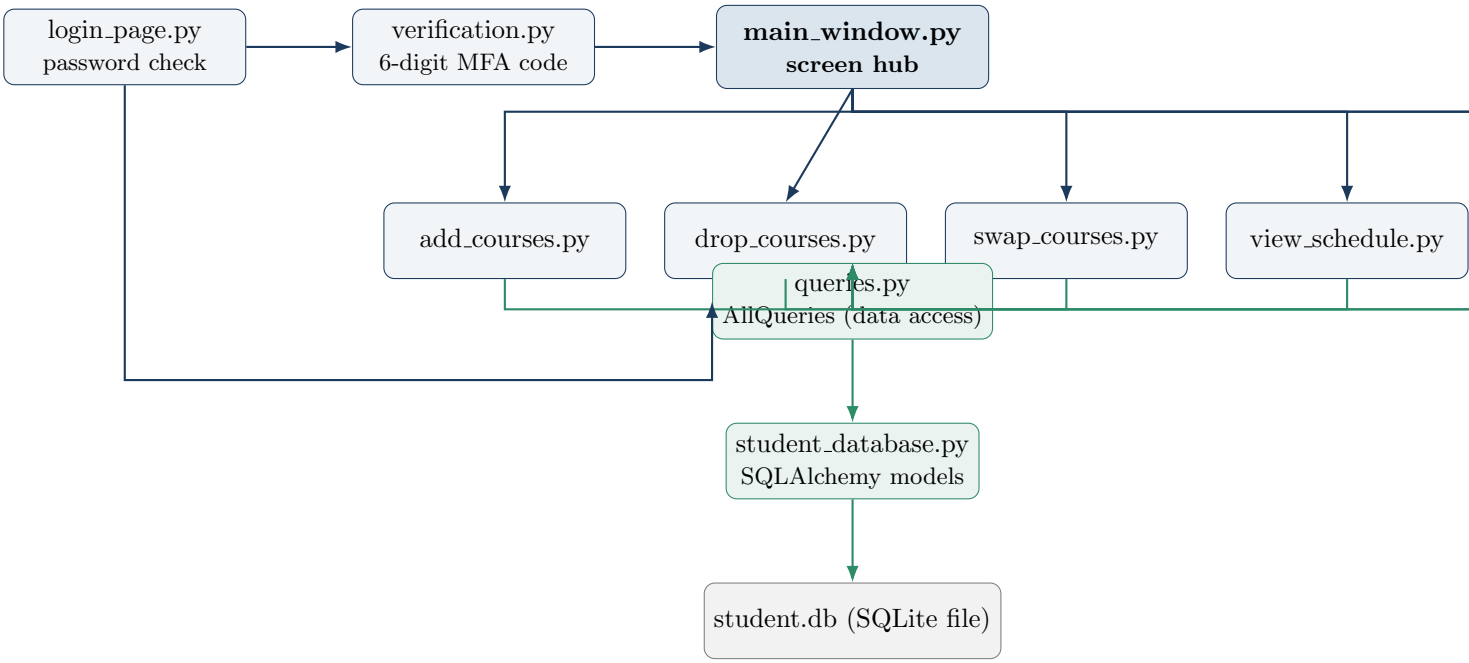
verification code and attendance-change confirmations through Gmail's SMTP server.

2 Directory layout

File (in `university-enrollment-system/`)	Role
`login_page.py`	entry point — password login
`verification.py`	MFA: emails/prints a 6-digit code
`main_window.py`	hub window, launches every other screen
`add_courses.py`	browse catalog, build a cart
`enroll.py`	confirm cart, finalize enrollment
`drop_courses.py`	drop an enrolled section
`swap_courses.py`	exchange one section for another
`view_schedule.py`	day/week schedule view, Excel export
`attendance.py`	mark attendance, email confirmation
`activity_log.py`	read-only history table
`queries.py`	`AllQueries`: the data-access layer
`student_database.py`	SQLAlchemy models + engine
`courses_database.py`	legacy seeding script, unused (Section 4)
eleven `.ui` files	Qt Designer layouts; ten loaded, one unused
`student.db`	pre-seeded SQLite database file
`Dockerfile` + 3 run scripts	containerized launch (Section 15)
`requirements.txt`, `.env.example`, `README.md`, `LICENSE`, `Proposal.pdf`	project metadata and docs

Every screen file (`login_page.py`, `add_courses.py`, etc.) follows the same pattern: it is a Python class inheriting from PyQt6's `QMainWindow`, and its visual layout is not built in Python at all but loaded at runtime from a matching `.ui` file — an XML file produced by the separate ***Qt Designer*** tool, which lets a developer drag-and-drop widgets instead of writing layout code. The call `uic.loadUi('screen.ui', self)` reads that XML and attaches every named widget (buttons, tables, labels) directly onto the Python object as attributes, e.g. a button named `login_button` in the `.ui` file becomes `self.login_button` in Python. [**flagged**] because this path is relative, every screen only loads correctly if the process's working directory is the project root; the README documents this explicitly as a known constraint (see Section 17).

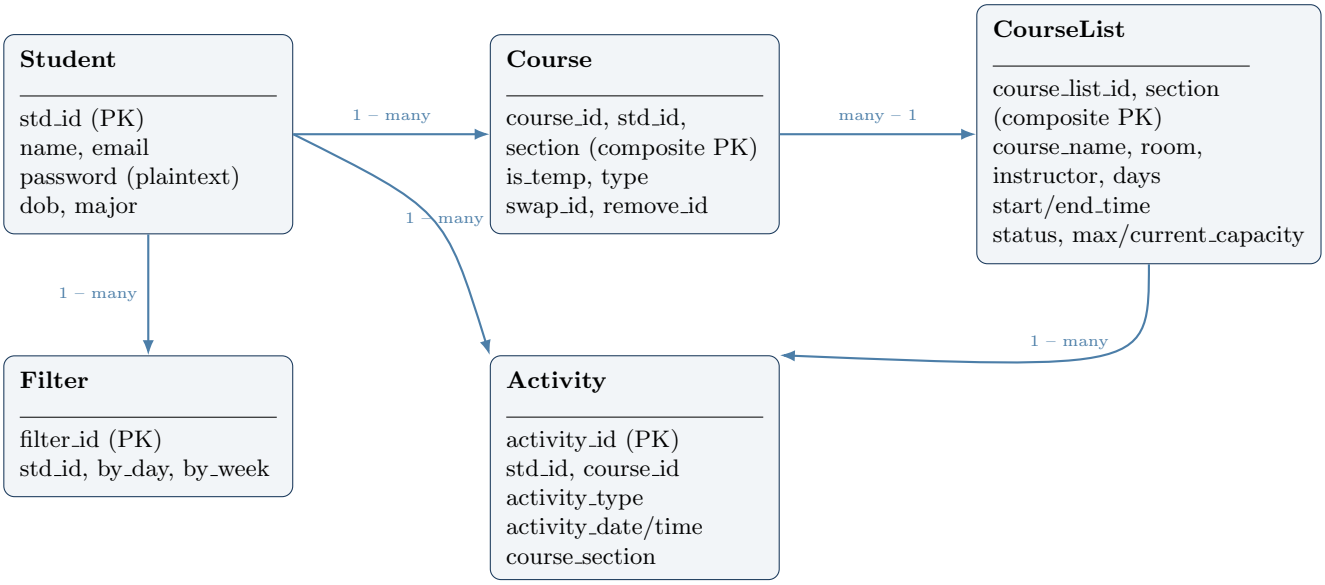
3 High-level architecture



Reading this diagram: the top row is the login sequence. `login_page.py` checks a plaintext password (Section 14) and, if correct, opens `verification.py`, which emails (or, without SMTP configured, prints) a 6-digit code; entering it correctly opens `main_window.py`. That hub window is a small dashboard with one button per feature; every button opens one of the six screens shown in the middle row. Every screen — and `login_page.py` itself, for the password lookup — calls into `queries.py`'s `AllQueries` class, which is the single point of contact with the database. `AllQueries` uses the SQLAlchemy models defined in `student_database.py`, which in turn read and write the `student.db` SQLite file. This `AllQueries`-as-sole-gateway shape is a form of the ***data access layer*** pattern: UI code never runs SQL directly, it only calls named methods like `add_course` or `get_all_courses`.

4 The data model

`student_database.py` defines five SQLAlchemy model classes, each mapping to one SQLite table via ***declarative mapping*** (the class body's `Column(...)` attributes describe a table's columns, and `relationship(...)` attributes describe foreign-key links, all without hand-written `CREATE TABLE` SQL).



What each table is for:

- `Student` is one row per student account. The `password` column is a plain `String` — there is no hashing (see Section 14).
- `CourseList` is the *course catalog*: one row per offered section (a specific course, taught by a specific instructor, at a specific time and room), with a `max_capacity`/`current_capacity` pair and a `status` boolean flag that is `True` while the section still has open seats.
- `Course` is the join between a student and a catalog section: it represents "this student is associated with this section." The `is_temp` boolean and `type` string ("Add", etc.) are how a section is marked as sitting in a student's *cart* versus being a confirmed enrollment (see Section 8). The `swap_id`/`remove_id` foreign keys are declared but — based on a full read of every query in `queries.py` — are never populated or read by any method; **[flagged]** these two columns are dead schema, most likely reserved for an originally-planned swap-tracking design that the final `swap_courses` method (Section 10) does not use.
- `Filter` stores a per-student day/week toggle. **[flagged]** by grepping every source file, no code anywhere constructs, queries, or reads a `Filter` row; the schedule screen's day/week toggle (Section 11) is implemented purely with an in-memory Python attribute (`self.filter_mode`), not this table. The `Filter` table appears to be unused, legacy schema.
- `Activity` is the log backing the activity-log screen: one row per add, drop, swap, filter-change, export, or attendance action, each stamped with a date and time.

A subtlety worth naming precisely: SQLAlchemy's ***declarative base*** is the common parent class (`Base = declarative_base()`) that every model class inherits from; it is what lets `Base.metadata.create_all(eng` discover all five model classes and create their tables if they don't already exist. ***primary key (PK)*** means the column(s) that uniquely identify a row; ***composite primary key*** means more than one column together forms that unique identity (`Course` needs `course_id + std_id + section` together because one student can have several sections of different courses, and one course has several sections).

4.1 courses_database.py — a second, unused model

The repository also contains `courses_database.py`, which defines its own `Student` class against the same `student.db` file, but with a different, incompatible schema (`id` instead of `std_id`, `dob` as a `String` instead of a `Date`). [**flagged**] A search of every `.py` file in the project shows nothing imports this module; it also never calls `create_all`, so it does not actually alter the database when it does run. Read on its own it looks like an earlier scratch file for seeding the course catalog, superseded by `student_database.py` and left in the repository rather than deleted. It has no effect on the running application.

Similarly, [**flagged**] the repository ships eleven `.ui` layout files, but a search of every `.py` file for `uic.loadUi` calls turns up only ten distinct filenames actually loaded (one per screen class described in this document). `add_remove_interaction.ui` is never referenced by any `.py` file; like `courses_database.py`, it appears to be a leftover from an earlier design (its name suggests a combined add/remove widget that was later split into the separate `add_courses.py`/`drop_courses.py` screens) and has no effect on the running application.

5 The data-access layer: queries.py

Every screen talks to the database exclusively through one class, `AllQueries`, instantiated fresh in almost every screen's `__init__`. It exposes about 30 methods such as `get_all_courses`, `add_course`, `remove_course`, `update_current_capacity`, and `swap_courses`, each wrapping one or a handful of SQLAlchemy queries against the shared, module-level `session` object imported from `student_database.py`.

5.1 The glob_id pattern (and why the README flags it)

`AllQueries.__init__` sets `self.glob_id = 20180`, a hardcoded default student id. Nearly every query method reads `std_id = self.glob_id` instead of taking a student id as a parameter. Each screen, on construction, calls `self.all_queries.update_std_id(self.all_queries.get_std_id_by_email(self.email))` to overwrite `glob_id` with the logged-in student's real id before doing anything else.

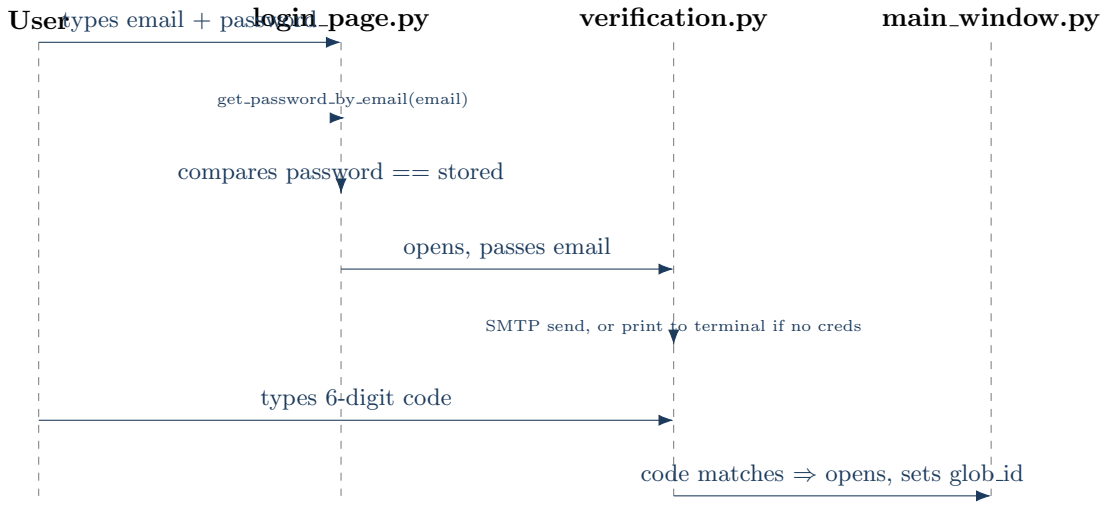
This works only because, in practice, one screen is open at a time and each screen updates `glob_id` as soon as it opens. But because *every* `AllQueries` instance shares the same underlying `Student`/`Course` rows (they all point at the same SQLite file and the same imported `session`), and because `glob_id` lives on each individual `AllQueries` *instance* rather than being passed explicitly, the correctness of every query depends on which window last called `update_std_id`. The project's own README names this directly as a lesson learned: "*Shared mutable state across screens is a bug source*," and lists "inject a session object owned by the main window" as a concrete fix that was not implemented. This document repeats that assessment because it is the single most important architectural weakness to be able to explain if asked about this codebase in an interview.

5.2 Side effects at import time

`student_database.py` does four things the moment it is imported, before any function is called: it creates the SQLAlchemy `engine` with `echo=True` (meaning *every* SQL statement SQLAlchemy issues is printed to the console for the rest of the program's life), calls `Base.metadata.create_all(engine)` (creating any missing tables), opens a `Session`, and then immediately closes it again (the module-level `session` object used everywhere else is a different, already-open session created by `sessionmaker()()` on the line above). Because Python only runs a module's top-level code once per process, this happens exactly once per app run, but it means simply writing `import student_database` (which every screen does, transitively, via `queries.py`) touches the real database file and starts SQL logging as a side effect

— there is no way to import the models without also connecting to the database. The README lists moving this behind a function or app entry point as a specific improvement that was not made.

6 Login and multi-factor verification



`login_page.py` reads the email and password text boxes and calls `AllQueries.get_password_by_email`. If the returned password string matches the typed one *exactly* (a plain `==` string comparison, not a cryptographic check of any kind), it opens `VerificationForm` from `verification.py` and hides itself. A wrong email or password shows a `QMessageBox.critical` error dialog — a modal pop-up window used throughout every screen for warnings and errors.

`VerificationForm.__init__` immediately generates a random six-digit number (`str(random.randint(100000, 999999))`) and calls `send_verification_email`. That method reads two environment variables, `ATTENDANCE_SENDER_EMAIL` and `ATTENDANCE_SENDER_PASSWORD` (the same two variables the attendance module uses, Section 12). If either is empty, it does not attempt any network call at all: it prints the code to the terminal and shows an informational dialog explaining that email is not configured. Only if both are set does it open an SMTP connection to `smtp.gmail.com:587`, start TLS encryption (***TLS***: Transport Layer Security, the encryption layer that keeps the login credentials used to authenticate with Gmail from being sent in the clear), log in with the sender credentials, and send the code as a plain-text email.

When the user submits the code they received (or read from the terminal), it is compared to the generated one. On a match, `MainStudentWindow` is constructed, and — this is the point where the shared `glob_id` first gets set for the session (Section 5) — `self.all_queries.glob_id` is overwritten with the real student id looked up by email.

Why this design

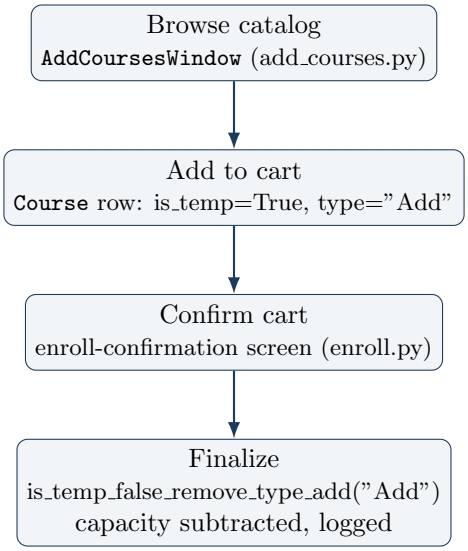
The README frames the email-optional design explicitly as solving a real constraint: this project is meant to be cloned and run by people (graders, portfolio visitors) who do not have the author's Gmail App Password. Gating the SMTP call on an environment-variable check, rather than attempting to send and catching failures after the fact, keeps the login flow honest — the user is told plainly that no email went out, rather than the app silently hanging on a network call that would never succeed without credentials, or falsely implying an email was sent.

7 The main window (hub)

`main_window.py`'s `MainStudentWindow` is a thin dispatcher: it stores `self.email` and looks up `self.std_id`, and every one of its six buttons (`view_attendance`, `add_courses_button`, `drop_courses_button`, `view_schedule_but` `swap_course_button`, `activity_log_button`, plus `logout_button`) is wired to a method that constructs the corresponding screen class, shows it, and hides the main window. Every child screen defines a custom PyQt ***signal*** named `closed` (`closed = pyqtSignal()`) — a signal is Qt's mechanism for one object to broadcast "something happened" to any number of listeners without either side needing to know about the other directly (the underlying ***observer pattern***). Each screen's overridden `closeEvent` emits `closed` when its window-manager close button ("X") is clicked, and `MainStudentWindow` connects that signal back to its own `reopen_login` method, so closing any screen returns the user to the hub. The `logout` button follows a slightly different path: it sets a `should_go_back` flag and calls `closeEvent` directly, which then emits the main window's own `closed` signal outward to `LoginForm`'s `reopen_login`, returning all the way to the login screen and clearing its text boxes.

8 Enrolling in courses: the cart pattern

Enrollment is split across two files that, confusingly, both define classes doing different jobs. **[flagged as a genuine naming ambiguity in the source]**: `add_courses.py` defines `AddCoursesWindow` (the course-browsing "cart" screen, loading `add_courses.ui`), and it imports a class also called `ViewScheduleWindow` from `enroll.py` for its "Next" step — but a *second*, entirely different class also named `ViewScheduleWindow` is defined in `view_schedule.py` (loading `view_schedule.ui`), which is the actual day/week schedule viewer reachable from the main hub (Section 11). These are two unrelated classes that happen to share a name because they live in different modules and are never imported into the same file together, so Python never raises a name collision — but a reader following only class names, rather than which file each `import` statement points at, will confuse the two. This document refers to them as *the enroll-confirmation screen* (`enroll.py`) and *the schedule-viewer screen* (`view_schedule.py`) to keep them distinct.



Clicking **Add** on a highlighted catalog row in `AddCoursesWindow` calls `can_the_course_be_added` (the schedule-clash check, described next), and if it passes, `AllQueries.add_course` inserts a new `Course` row for the current student with `is_temp=True` and `type="Add"` — this is the cart: a provisional row that does not yet count as an enrollment and has not yet touched the section's `current_capacity`. The row is mirrored into a second on-screen table (`show_selected_schedule_widget`), and clicking

Remove deletes it back out with `remove_course`. Clicking **Next/Enroll** opens the enroll-confirmation screen, which lists every cart row; clicking its own **Enroll** button first checks that no listed section shows "Closed" status, then calls `is_temp_false_remove_type_add("Add")`, which flips every `is_temp` cart row for this student to `False` and clears `type` — this is the moment a cart item becomes a real enrollment — then loops over every enrolled section subtracting one from `current_capacity` (and setting `status=False` once a section hits zero seats), and finally writes one `Activity` row logging the whole batch of course ids and sections as a single "Enroll" entry.

8.1 The clash-detection check

Both the add-course flow and the swap-course flow (Section 10) need to answer the same question: *does this new section's meeting time overlap with anything the student is already committed to?* Both implement essentially the same algorithm independently (`can_the_course_be_added` appears, with minor differences, in both `add_courses.py` and `swap_courses.py`); it is written out once here.

```
function can_the_course_be_added(course_id, start, end, days):
  existing ← every currently-enrolled section's (start, end, days)
  new_start ← start converted to seconds since midnight
  new_end ← end converted to seconds since midnight
  for each (e_start, e_end, e_days) in existing:
    if new section and existing section share any weekday letter-pair:
      if e_start ≤ new_start ≤ e_end:  clash
      if e_start ≤ new_end ≤ e_end:  clash
      if new_start ≤ e_start and new_end ≥ e_end:  clash
  return no clash found
```

Two implementation details matter here. First, days are stored as a single string like `"MoWeFr"` (two-letter codes concatenated); `get_days_list` splits this back into pairs with Python slice stepping (`[days [x:x+2] for x in range(0, len(days), 2)]`) so two sections can be compared day-by-day. Second, `time_to_seconds` converts a Python `time` object (hour/minute/second) into a single integer count of seconds since midnight, which is what makes the three numeric comparisons above possible — times cannot be compared for "does this range overlap that range" as cleanly in their original form. The three `if` conditions cover, respectively: the new section *starts* inside an existing block, the new section *ends* inside an existing block, and the new section fully *contains* an existing block — together these three checks catch every way two time ranges can overlap.

9 Dropping a course

`drop_courses.py`'s `DropCoursesWindow` lists every section the student is currently, non-provisionally enrolled in (`get_all_courses_by_id`, which filters `is_temp=False` — recall provisional cart rows have `is_temp=True` and would not appear here). Selecting a row and clicking **Drop** raises a `QMessageBox.question` confirmation dialog (Yes/No); on Yes, it deletes the `Course` row via `remove_course`, adds one back to the section's `current_capacity` (re-opening it if it had been full), removes the row from the on-screen table, and logs a "Drop" activity entry.

10 Swapping a course

`swap_courses.py`'s `SwapCoursesWindow` shows two tables: the full catalog on the left (excluding sections the student is already enrolled in) and, after selecting a catalog row and clicking **Select**, every section of *that same course* the student is currently enrolled in on the right. Clicking **Swap** runs the

clash check against the newly selected section (excluding the section being replaced from the clash comparison, via `get_timings_of_all_courses_by_id`'s exclude parameters), asks for confirmation, and then calls `AllQueries.swap_courses`. That method is a sequence of three steps run one after another — remove the old `Course` row, add the new one, and mark the new one `is_temp=False` — *not* a single atomic database transaction: if the middle step (adding the new section) fails after the old one has already been removed, the method returns `False` but the student is left with neither course. **[flagged]** this is a real correctness gap: nothing in `swap_courses` rolls the deletion back if the subsequent add fails. On success, the UI updates both tables, adjusts `current_capacity` on both the old and new sections, and logs a single "Swap" activity entry recording both course/section values as an arrow-joined string (e.g. `"CS101 -> CS102"`).

11 Viewing and exporting the schedule

`view_schedule.py`'s screen offers two view modes, tracked purely by an in-memory string attribute `self.filter_mode` (`"Day"` or `"Week"` — not backed by the unused `Filter` database table, see Section 4):

- **Day mode** (the default) shows only sections that meet on *today's* weekday, computed via Python's `datetime.now().strftime("%A")`, with an extra "Current Day" column.
- **Week mode** shows every enrolled section, hides the "Current Day" column, and enables export.

Clicking **Export** while in Week mode builds a ***pandas*** `DataFrame` (pandas is a Python library for working with tabular data, similar in spirit to a spreadsheet inside code) from the visible table cells and writes it to `schedule_by_week.xlsx` using `DataFrame.to_excel`, which relies on the `openpyxl` library to produce a real `.xlsx` file. `save_data` avoids silently overwriting a previous export: it first tries `pandas.read_excel` on the target path, and if that succeeds (the file already exists), it retries with an incremented filename (`schedule_by_week (1).xlsx`, `(2)`, and so on) until `read_excel` raises `FileNotFoundError` at an unused name, at which point it writes there. Exporting while in **Day mode** is deliberately disabled and always shows an "Export Not Available" dialog: the README explains that day mode hides a column, which previously caused the exported columns and data to misalign, so the feature was turned off outright rather than shipped broken. Every filter switch and every export call also writes a row to the activity log.

12 Attendance

`attendance.py`'s `AttendanceForm` is opened from the main hub with the logged-in student's email and id. Its table is populated from a **hardcoded Python list of four dummy rows** (dates in March 2025, alternating Present/Absent) — it does not read from any database table, and there is no `Attendance` model in `student_database.py` at all. Each row's status is an editable `QComboBox` (Present/Absent dropdown) rather than a plain, read-only cell. Clicking **Mark Attendance** loops over every row, reads the combo box's current selection, and for each row: (1) attempts to email a confirmation via `send_attendance_email` (same environment-variable-gated SMTP pattern as Section 6, returning `False` rather than raising if credentials are absent) and (2) writes an `Activity` row via `AllQueries.insert_activity` describing the change in plain text (e.g. `"Attendance for Monday (2025-03-01) changed to Present"`). If any email failed to send, the closing dialog says so honestly ("confirmation emails could not be sent") rather than claiming success. **[flagged]** because the underlying table data is not persisted anywhere, re-opening the attendance screen always shows the same four dummy rows again, regardless of what was previously marked — the change only survives in the activity log's text description, not as queryable attendance state. The README lists this as a

known, intentional-for-scope limitation ("the attendance table is dummy data ... It should have been an Attendance model like everything else").

13 Activity log

`activity_log.py`'s `ActivityLogWindow` is the simplest screen: it calls `AllQueries.get_activity_by_std_id` (every `Activity` row for the current student, unfiltered and unsorted — in whatever order SQLAlchemy's default query returns them) and renders it into a six-column, read-only table: activity id, activity type, course id, section, date, time. Every other screen in the app writes into this same table as a side effect of its own actions — there is no separate "logging service"; each screen calls `add_data_to_activity_log` or one of `insert_activity`/ `insert_attendance_activity` directly at the point where the user-visible action happens.

14 Security posture (read this before demoing the app)

This section consolidates every security-relevant fact already touched on above, because it is the material most likely to come up in a defense or interview.

- **Passwords are stored and compared in plaintext.** `Student.password` is a plain SQLAlchemy `String` column, and `login_page.py` compares the typed password to the stored one with a bare `==`. There is no hashing (e.g. bcrypt) anywhere in the codebase. The README states this directly and calls it "a real weakness, not a design choice worth defending," and lists bcrypt hashing under "What I'd do differently."
- **The seed accounts are fake, demo-only credentials**, documented in the README (`bob123@st.habib.edu.p` etc.) purely so a cloned copy of the repo is immediately usable; they are explicitly flagged as not to be reused with real credentials.
- **SMTP credentials are never hardcoded.** Both the verification and attendance email paths read `ATTENDANCE_SENDER_EMAIL` / `ATTENDANCE_SENDER_PASSWORD` from the environment (documented in `.env.example`) and explicitly skip the network call rather than sending with blank/placeholder credentials.
- **No SQL injection surface found.** Every database query in `queries.py` goes through SQLAlchemy's ORM query methods (`session.query(...).filter_by(...)`), which parameterize values automatically; the codebase does not construct raw SQL strings from user input anywhere.

15 Packaging: Docker and the per-OS run scripts

The `Dockerfile` builds from `python:3.10-slim`, installs the system libraries PyQt6 needs to render a GUI at all even inside a container (X11 client libraries such as `libx11-6`, plus `libgl1-mesa-glx` for OpenGL support Qt requires), installs the Python dependencies from `requirements.txt`, copies the whole project in (including the pre-seeded `student.db`), sets `QT_QPA_PLATFORM=xcb` (the Qt ***platform plugin*** that tells Qt to render through the X Window System rather than, say, headless offscreen rendering), and runs `python login_page.py` as its entrypoint. Because PyQt6 needs an actual display surface to draw windows onto, and a Docker container has no display of its own, the three run scripts (`run-linux.sh`, `run-macos.sh`, `run-windows.ps1`) each forward the *host* machine's display into the container: on Linux by bind-mounting the X11 Unix socket (`/tmp/.X11-unix`) and setting `DISPLAY`; on macOS via XQuartz and the host's network interface IP; on Windows via an X server such as VcXsrv and `host.docker.internal`. All three scripts build a local image tagged `pscs-app` if one is not

already present — the tag name is a direct reference to the PSCS platform this project is prototyping improvements for (Section 1 above).

16 Testing — what the README claims versus what is in the repository

The README's "Challenges" and "What I learned" sections describe, in some detail, a headless smoke-testing approach: setting the environment variable `QT_QPA_PLATFORM=offscreen` so PyQt6 can run without a real display, and monkey-patching (temporarily replacing at runtime) the static `QMessageBox` methods (`information`, `warning`, `critical`) so they record their calls instead of blocking on a modal dialog that nothing would click in an automated run. It states this was used to build and query every screen in one pass and that "this is how the project was last verified." [**flagged**] A full listing of the repository (every file, not only `.py` files) turned up no test file of any kind — no `test_*.py`, no `tests/` directory, no CI configuration. The testing methodology the README describes is therefore either not currently present in this copy of the repository (possibly run ad hoc and never committed) or was removed at some point; this document cannot confirm which from the source alone, and states this honestly rather than asserting the smoke test still exists.

17 Limitations and stated future work

The following is drawn directly from the README's "What I'd do differently" section, kept here verbatim in substance so nothing is invented beyond what the project's own authors documented:

- Hash passwords (bcrypt) instead of storing and comparing plaintext.
- Replace the mutable, shared `AllQueries.glob_id` with a session object owned by the main window and passed explicitly to every screen.
- Use one shared `AllQueries` instance and one DB session across the whole app, injected into screens, instead of each window constructing its own.
- Give attendance a real `Attendance` database model instead of hardcoded dummy data, so marks persist between sessions.
- Replace scattered `print` debugging with proper logging, and stop `student_database.py` from running `create_all` and SQL-echoing as an import side effect.
- Add automated tests; the query layer is separable enough to have had unit tests from the start.

Also worth naming, found during this review and not already in the README: the `swap_courses` data method is not transactional (Section 10), the `Course.swap_id`/`remove_id` columns and the entire `Filter` table are unused schema (Section 4), and `courses_database.py` is a dead, unimported legacy file.

18 Glossary

- **GUI** — Graphical User Interface; windows, buttons, and controls a user interacts with visually, as opposed to a text-only command line.
- **PyQt6** — Python bindings for Qt6, a C++ GUI toolkit; lets Python code build native-looking desktop windows.

- **Qt Designer / `.ui` file** — a visual tool for laying out a window's widgets, saved as an XML file and loaded at runtime rather than hand-coded.
- **ORM** — Object-Relational Mapper; a library (here, SQLAlchemy) that lets code treat database rows as ordinary objects.
- **SQLite** — a serverless, single-file relational database engine.
- **Declarative base / model class** — in SQLAlchemy, a Python class whose attributes describe a database table's columns and relationships.
- **Primary key / composite primary key** — the column(s) that uniquely identify a row; composite means more than one column is needed together.
- **Foreign key** — a column whose value must match a primary key in another table, expressing a link between two rows.
- **SMTP** — Simple Mail Transfer Protocol; the network protocol used to send email, used here via Python's `smtplib`.
- **TLS** — Transport Layer Security; the encryption used to protect the SMTP login and message from being read in transit.
- **MFA** — Multi-Factor Authentication; requiring more than one proof of identity (here, a password plus an emailed one-time code) to log in.
- **Signal (Qt)** — a mechanism for one object to announce an event to any number of listeners without either side needing a direct reference to the other.
- **Data-access layer** — a single class or module (here, `AllQueries`) through which all other code must go to read or write the database, keeping SQL out of the UI code.
- **pandas / DataFrame** — a Python library for tabular data; a `DataFrame` is its in-memory table object, here converted directly to an Excel file.
- **Monkey-patching** — temporarily replacing a function or method at runtime (e.g. for testing), rather than editing its source definition.